

Introduction

Dans le cadre du module de Protocoles Réseaux, il nous a été demandé de concevoir et de simuler un réseau local structuré en anneau, en nous inspirant fortement du protocole Token Ring (IEEE 802.5).

L'objectif principal de ce projet est de faire communiquer plusieurs machines entre elles de manière ordonnée et sans collision. Dans ce modèle, la règle est simple : une machine ne peut émettre des données sur le réseau que si elle possède un "jeton" unique qui circule de nœud en nœud.

Pour répondre aux contraintes du cahier des charges, notre réseau devait être dynamique. Cela signifie qu'une nouvelle machine doit pouvoir s'insérer dans l'anneau à tout moment, et une machine existante doit pouvoir le quitter proprement, le tout sans briser la chaîne de communication ni perdre le jeton. De plus, le réseau doit permettre l'envoi de messages ainsi que le transfert fiable de fichiers.

Afin de séparer la logique réseau de l'interface utilisateur, nous avons modélisé chaque machine à l'aide de deux processus indépendants développés en langage C :

1. Le processus Driver : Il agit comme un routeur en tâche de fond. Il écoute en continu les paquets provenant de son voisin de gauche via une socket TCP, gère le passage du jeton, et fait suivre les données à son voisin de droite.
2. Le processus Comm : C'est l'interface avec laquelle l'utilisateur interagit. Il communique avec le Driver via une socket locale pour lui donner l'ordre d'envoyer des messages, des fichiers, ou de récupérer la topologie du réseau.

Ce compte-rendu détaille nos choix d'implémentation, les structures de données utilisées pour le routage, ainsi que les solutions que nous avons mises en place pour gérer la dynamique de l'anneau et les éventuelles pannes.

Partie 1 : Préliminaires et Étude Théorique

Avant de développer notre propre version de l'anneau, nous avons étudié le protocole historique dont il s'inspire : le Token Ring, ainsi que ses alternatives et ses faiblesses.

1. Analyse du protocole Token Ring (IEEE 802.5)

Le protocole Token Ring repose sur une topologie réseau où chaque machine est reliée à un voisin de gauche et un voisin de droite, formant une boucle fermée. La communication est unidirectionnelle.

Le principe fondamental est le suivant : un petit paquet de contrôle, circule indéfiniment d'une machine à l'autre.

- Si une machine reçoit le jeton et n'a rien à dire, elle le fait simplement suivre à son voisin.

- Si elle veut envoyer un message, elle doit attendre de recevoir ce jeton. Elle le "capture", insère ses données, et envoie le tout sur l'anneau. Le message fait le tour jusqu'au destinataire, puis revient à l'émetteur qui libère alors un nouveau jeton.

L'énorme avantage de cette méthode est qu'elle est déterministe : on peut calculer le temps maximum qu'une machine devra attendre avant de pouvoir parler. Il n'y a donc aucune collision de données possible.

2. Comparaison avec Ethernet

Pour bien comprendre l'utilité du Token Ring, il faut le comparer à Ethernet, le standard réseau le plus connu.

- Le fonctionnement : Contrairement à l'anneau où l'on attend son tour, Ethernet fonctionne comme une discussion de groupe. Une machine écoute le réseau. S'il y a du silence, elle parle. Si deux machines parlent exactement en même temps, il y a une "collision" de données. Les deux s'arrêtent, attendent un temps aléatoire, et réessaient. C'est une méthode probabiliste.
- Les performances : À faible charge, Ethernet est beaucoup plus rapide car on émet tout de suite, alors qu'en Token Ring il faut parfois attendre que le jeton arrive jusqu'à nous.

À forte charge, Ethernet s'effondre car il y a trop de collisions, ce qui bloque le réseau. À l'inverse, le Token Ring reste parfaitement stable, fluide et équitable puisque le jeton régule la parole.

3. Réflexion sur la tolérance aux pannes

La structure physique en anneau est élégante, mais elle est très fragile. Nous avons identifié deux problèmes majeurs auxquels un système distribué de ce type doit faire face :

1. La perte du jeton : Si la machine qui détient le jeton subit un crash matériel ou logiciel avant d'avoir pu le faire suivre, le jeton disparaît. Résultat : l'anneau tourne à vide indéfiniment et plus personne ne peut parler.
2. La rupture de l'anneau : Étant donné que le réseau forme une boucle en série, si le câble réseau entre deux machines est coupé, ou si une machine est débranchée brutalement, le circuit est "ouvert". Les messages partent dans le vide et la communication s'arrête totalement.

Dans la conception de notre projet, nous avons réfléchi à ces problèmes. Pour pallier la perte du jeton, nous avons intégré un mécanisme manuel permettant à n'importe quel nœud de réinjecter un jeton de secours. Pour limiter les ruptures d'anneau, nous avons implémenté une procédure de déconnexion "propre" où la machine qui souhaite partir prévient ses voisins pour qu'ils referment l'anneau avant qu'elle ne coupe sa connexion.

Partie 2 : Architecture et Conception

Pour simuler chaque machine de notre anneau de manière réaliste et modulaire, nous avons divisé le système en deux programmes distincts. Ce choix d'architecture permet de séparer complètement la logique de routage réseau de l'interaction avec l'utilisateur.

1. Le Modèle à Deux Processus

Chaque nœud du réseau est simulé par le lancement en parallèle de deux processus, correspondant à nos deux fichiers sources principaux :

- Le processus Driver (driver.c)
- Le processus Comm (comm.c)

Pour que ces deux programmes puissent s'échanger des informations, ils sont reliés par une socket locale TCP. Le processus Driver ouvre un port d'écoute local sur la machine, auquel le processus Comm vient se connecter au démarrage.

2. Le processus Driver : Le cœur du réseau

Le Driver agit comme le véritable routeur de notre nœud et fonctionne de manière transparente en tâche de fond. Sa responsabilité est de maintenir l'anneau et de faire circuler les données.

Pour accomplir sa tâche, le Driver gère trois descripteurs de sockets en permanence :

- sockg (Socket Gauche) : Utilisée pour recevoir les données provenant de la machine précédente dans l'anneau.
- sockd (Socket Droite) : Utilisée pour expédier les données vers la machine suivante.
- sockl (Socket Locale) : Utilisée pour communiquer avec l'interface Comm de l'utilisateur.

Choix technique majeur : Étant donné que le Driver doit écouter simultanément la gauche, la droite et le local sans jamais que le programme ne se bloque dans une fonction de lecture, nous avons utilisé `select()`. Cette fonction surveille tous nos descripteurs de fichiers et réveille le processus uniquement lorsqu'un événement survient sur l'une des sockets.

C'est également le Driver qui est le gardien du jeton. Lorsqu'il reçoit le jeton sur sockg, il vérifie si une donnée locale est en attente d'émission. Si c'est le cas, il injecte la donnée sur sockd. Sinon, il se contente de faire suivre le jeton à son voisin droit pour ne pas ralentir le réseau.

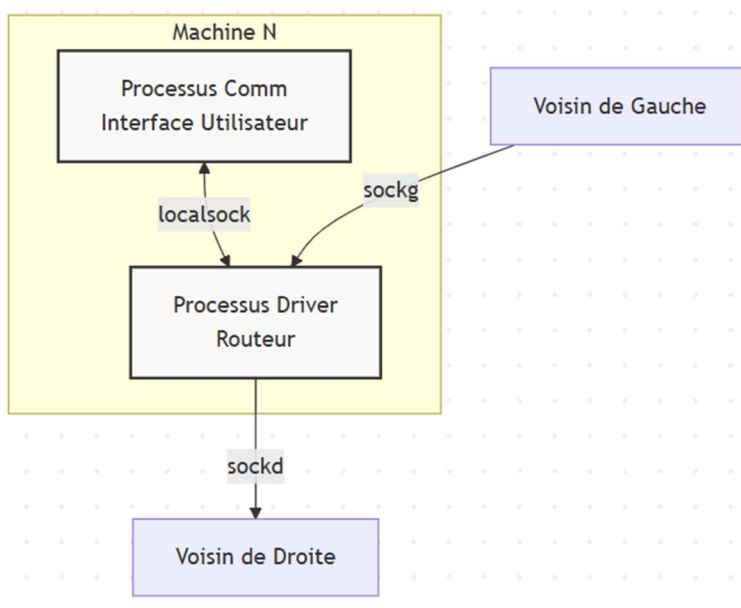


Figure 1 : Architecture interne d'un nœud et routage des communications locales et réseau.

3. Le processus Comm : L'Interface Homme-Machine

Le fichier comm.c gère l'interface avec laquelle l'utilisateur interagit. Il affiche un menu permettant de choisir entre 6 actions : Emettre un message, Diffuser, Afficher la topologie, Transférer un fichier, Régénérer le jeton ou Quitter.

Choix technique : Une contrainte importante de notre conception est que le processus Comm est totalement passif vis-à-vis du réseau externe. Il ne sait pas comment l'anneau est construit.

- En émission : Si l'utilisateur veut envoyer un message ou un fichier, le Comm lit les données, les formate proprement, et les pousse vers le Driver via la socket locale. Il délègue l'attente du jeton au Driver.
- En réception : Le Comm se contente d'écouter la socket locale. Lorsque le Driver intercepte un message ou un fichier dont l'adresse de destination correspond à notre machine, il le passe au Comm. Ce dernier s'occupe alors d'afficher le texte à l'écran de manière lisible, ou de créer un fichier sur le disque dur pour y sauvegarder les données binaires reçues.

Cette architecture garantit que si l'utilisateur met beaucoup de temps à taper un message ou à naviguer dans le menu, le réseau n'est absolument pas ralenti, car le Driver continue de faire tourner l'anneau en arrière-plan de manière autonome.

4. Fonctions utilitaires implémentées Pour gérer les communications et abstraire la complexité des sockets, nous avons développé plusieurs fonctions spécifiques dans nos deux processus :

- `obtenir_ip_locale()` : Permet de récupérer dynamiquement l'adresse IP de la machine hôte (via `gethostname` et `gethostbyname`) afin de signer les paquets émis.
- `se_connecter()` / `connecter_droit()` : Gèrent la résolution DNS et l'ouverture de la socket TCP vers le nœud suivant, en convertissant les adresses et les ports (via `htons`).
- `read_all()` / `read_ex()` : Ces fonctions sont cruciales. En TCP, la réception d'un paquet complet en un seul appel n'est pas garantie. Ces fonctions encapsulent un bloc `while` qui force le programme à lire le flux réseau jusqu'à avoir reçu exactement le nombre d'octets attendus pour reconstituer une structure Paquet complète, évitant ainsi les données tronquées.

Partie 3 : Spécifications Techniques et Algorithmes

Cette section détaille les choix d'implémentation que nous avons faits pour répondre aux exigences du cahier des charges, notamment sur le format des échanges, la gestion de la topologie et la fiabilité des transferts.

1. Structure de Données : Le Format des Messages

Pour que toutes les machines se comprennent, nous avons défini un protocole commun centralisé dans le fichier `protocole.h`. Toutes les communications transitent sous la forme d'une structure C appelée Paquet :

- `type (int)` : Permet au Driver de savoir comment router le paquet (ex: `TYPE_JETON`, `TYPE_MESSAGE`, `TYPE_FICHIER`, `TYPE_INFO...`).

- **taille (int)** : Indique la taille utile des données contenues dans le message. C'est une information pour délimiter les données binaires lors des transferts de fichiers.
- **id_dest et id_src** : Contiennent les adresses IP du destinataire et de l'émetteur, permettant au Driver de savoir si le paquet lui est destiné ou s'il doit le faire suivre.
- **buffer** : Contient le corps du message ou le fragment de fichier, limité par TAILLE_MAX (1024 octets).

2. Algorithme d'Insertion Dynamique

L'une des contraintes majeures était de permettre à une machine de joindre l'anneau à tout moment. Plutôt que de redémarrer le réseau, nous avons implémenté une insertion à la volée.

Voici la cinématique lorsqu'une nouvelle machine veut s'insérer à côté d'une machine existante :

1. La machine N se connecte à la machine A et s'identifie avec un message TYPE_HELLO.
2. La machine A accepte la connexion. N devient son nouveau voisin de gauche.
3. Pour que l'anneau soit refermé, la machine A demande à son ancien voisin de gauche (B) de se dérouter. A envoie un paquet TYPE_RECONFIG à B contenant l'adresse de N.
4. En recevant cet ordre, la machine B ferme sa connexion avec A et se connecte à N. L'anneau est mis à jour dynamiquement.

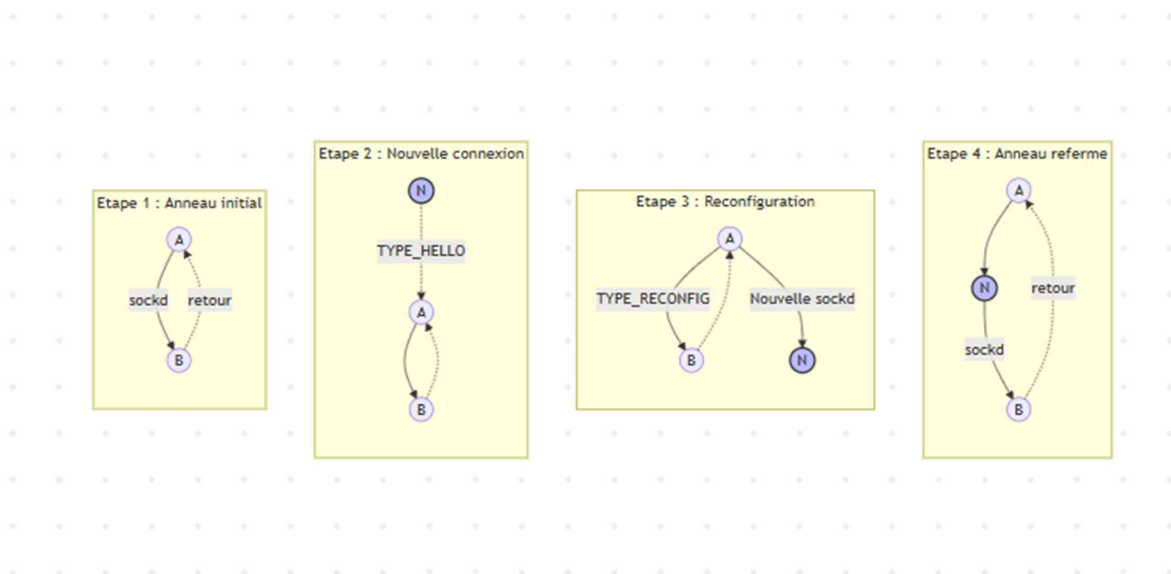


Figure 2 : Les étapes de reconfiguration réseau lors de l'insertion d'un nouveau nœud (N)

3. Récupération de la Topologie

Pour la fonction "Récupérer", le cahier des charges demande d'afficher la liste des machines avec leur Nom, Adresse, Port d'Entrée et Port de Sortie.

Plutôt que de "coder en dur" ces valeurs, nous avons utilisé une approche basée sur les primitives de l'API Sockets en C :

- Lorsqu'un utilisateur demande l'état du réseau, un paquet TYPE_INFO vide est injecté sur l'anneau.

- À chaque fois que ce paquet traverse un processus Driver, la machine y ajoute ses propres informations.
- Pour récupérer son nom, le programme utilise `gethostname()`.
- Pour les ports, le Driver interroge directement la mémoire du noyau Linux. Il utilise `getsockname()` pour extraire le port de sa socket d'écoute (Port Entrant) et `getpeername()` pour la socket connectée à son voisin (Port Sortant).
- Point d'attention : Les valeurs réseau étant stockées en Big-Endian, nous utilisons la fonction `ntohs()` pour convertir les ports et les afficher correctement.

Le paquet fait le tour de l'anneau, accumule les données sous la forme `Nom|IP|PortE|PortS`; et retourne à l'émetteur où l'interface Comm découpe la chaîne pour afficher un graphique textuel vertical.

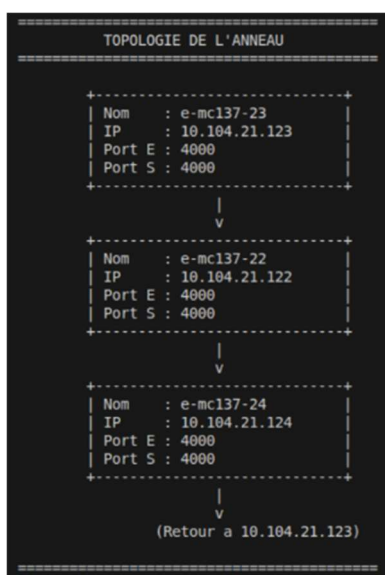


Figure 1 : Affichage dynamique de la topologie de l'anneau depuis le processus Comm.

4. Transfert de Fichier Fiable

Le transfert de fichiers a nécessité une réflexion particulière. Puisque la taille de notre buffer réseau est limitée à 1024 octets (`TAILLE_MAX`), il était impossible d'envoyer une image ou un fichier binaire lourd en un seul paquet.

Nous avons donc implémenté un algorithme de fragmentation logicielle directement dans le processus Comm, selon une machine à états très stricte basée sur la valeur de taille :

1. Signal de départ (taille = -1) : Le Comm émetteur prévient qu'un fichier arrive. Le Comm récepteur ouvre le fichier de destination en mode `wb`, ce qui garantit qu'on part d'un fichier vide.
2. Transfert des fragments (taille > 0) : Le fichier est lu par morceaux de 1024 octets avec `fread`. Chaque morceau est envoyé sur l'anneau. À la réception, les données sont collées à la suite du fichier grâce à l'ouverture en mode `ab`.

3. Signal de fin (taille = 0) : Le transfert se termine et le fichier est clôturé proprement.

Contrôle de flux : Pour ne pas écraser la mémoire du processus Driver, le processus Comm émetteur utilise un `usleep()` entre chaque fragment. Cette micro-pause laisse le temps au jeton de faire le tour de l'anneau pour livrer le paquet précédent. C'est une méthode de régulation de débit pour assurer la fiabilité du transfert.

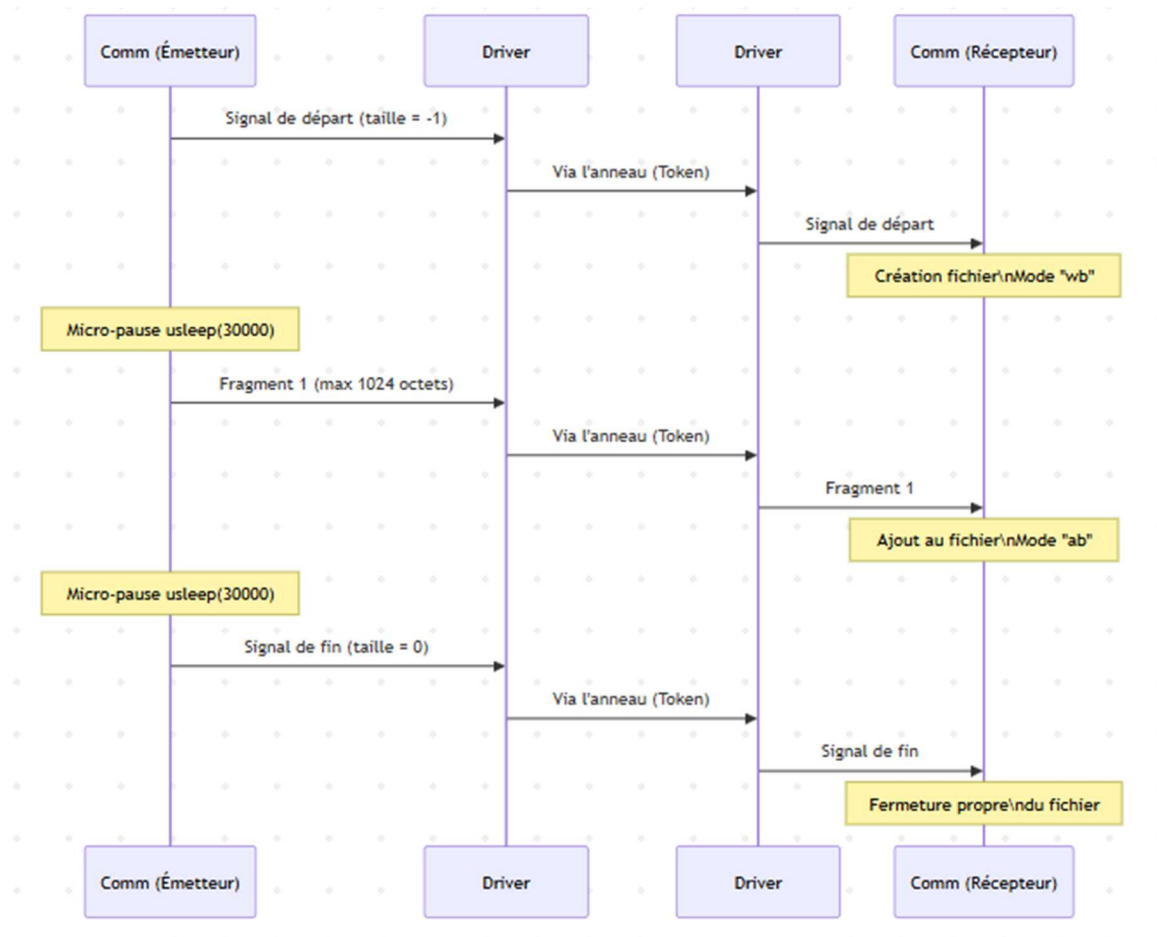


Figure 4 : Diagramme de séquence illustrant la fragmentation d'un fichier et la temporisation (usleep) pour respecter le passage du jeton.

Partie 4 : Gestion des Erreurs et Tolérance aux Pannes

Un système distribué en anneau est vulnérable. Si un seul maillon de la chaîne se casse, c'est l'ensemble du réseau qui est paralysé. Pour répondre aux problèmes de tolérance aux pannes, nous avons implémenté plusieurs mécanismes de sécurité au niveau du réseau et de l'application.

1. La déconnexion propre

Le premier risque est qu'une machine quitte l'anneau et laisse un "trou" derrière elle. Pour éviter cela, nous avons implémenté une procédure de départ coopérative via le message `TYPE_BYE`.

Lorsqu'un utilisateur choisit l'option "Quitter" dans son interface Comm :

1. Le nœud qui souhaite partir crée un paquet TYPE_BYE. Il met sa propre adresse IP en expéditeur, et il place dans le corps du message les coordonnées exactes de son voisin de droite actuel.
2. Il envoie ce paquet sur l'anneau.
3. Lorsque le voisin de gauche reçoit ce paquet TYPE_BYE, il lit le buffer, comprend que sa cible actuelle s'en va, et récupère les nouvelles coordonnées.
4. Il ferme alors sa socket droite actuelle et se reconnecte automatiquement au nouveau voisin.

Grâce à ce mécanisme, l'anneau "cicatrise" et se referme de lui-même avant même que la machine sortante n'ait complètement coupé son programme.

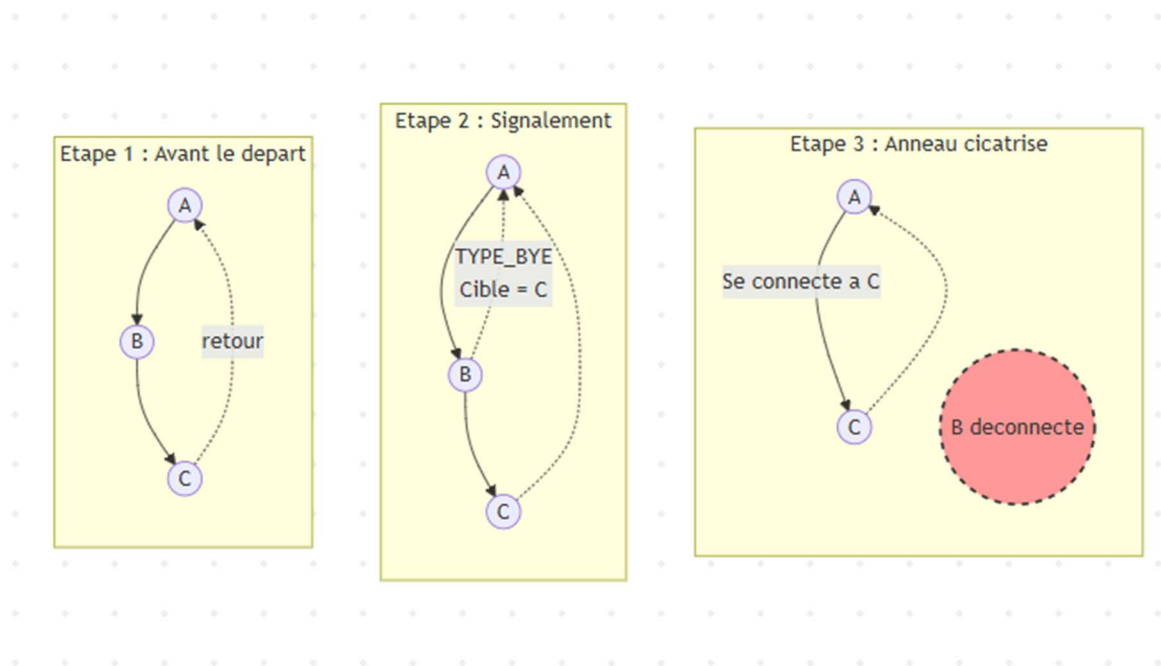


Figure 5 : Cicatrisation de l'anneau suite à l'émission d'un paquet TYPE_BYE par la machine B.

2. La prévention des crashes silencieux (SIGPIPE)

Dans la réalité, une machine ne prévient pas toujours avant de partir. En langage C, si le processus Driver tente d'écrire sur la socket d'un voisin qui vient de se déconnecter brutalement, le système d'exploitation Linux envoie un signal SIGPIPE. Par défaut, ce signal tue instantanément le programme, provoquant une réaction en chaîne qui détruit tout l'anneau.

Pour rendre notre Driver robuste :

- Nous avons utilisé l'instruction `signal(SIGPIPE, SIG_IGN)`; au tout début de notre `main()`. Cela indique au noyau Linux d'ignorer cette erreur fatale.
- En parallèle, notre boucle d'écoute `select()` vérifie en permanence la valeur de retour de la fonction de lecture. Si `read <= 0`, le Driver détecte immédiatement que le voisin de gauche a disparu et ferme proprement le descripteur de fichier, évitant ainsi de boucler dans le vide et de faire surchauffer le processeur.

3. La régénération du jeton en cas de perte

L'autre grande faille du protocole Token Ring est la perte du jeton. Si la machine qui détient physiquement le jeton dans sa mémoire vive crashe avant d'avoir pu le transmettre à son voisin, le jeton est définitivement détruit. Plus aucune machine ne recevra le droit de parler, et le réseau restera figé pour l'éternité.

Comme il est très complexe de faire élire automatiquement un nouveau jeton par les machines restantes sans créer de "doublons, nous avons opté pour une solution intégrée à l'interaction : le mode SOS.

L'option 5 du menu Comm permet à n'importe quel utilisateur de forcer la création d'un nouveau paquet TYPE_JETON et de l'injecter sur l'anneau. Cela permet de débloquent immédiatement la situation et de relancer le cycle de communication normal après une panne.

Partie 5 : Problèmes à discuter et Réflexions

Le cahier des charges nous invitait à réfléchir sur l'efficacité de notre modèle, notamment concernant l'envoi de données lourdes.

1. La structure d'anneau est-elle bien adaptée aux transferts de fichiers ?

Non, elle ne l'est pas. Si la structure en anneau à jeton est excellente pour échanger des messages textes courts de manière équitable, elle devient très inefficace pour le transfert de gros fichiers.

La raison principale est la difficulté du jeton. Dans notre implémentation, la taille maximale d'un paquet est de 1024 octets. Pour envoyer un fichier de seulement 2 Mo, il faut le découper en près de 2000 fragments. Or, sur un réseau Token Ring, une machine ne peut envoyer qu'un seul paquet à chaque fois qu'elle reçoit le jeton. Pour envoyer notre fichier de 2 Mo, l'émetteur doit capturer le jeton 2000 fois.

C'est exactement pour cette raison que nous avons dû intégrer la fonction `usleep(30000)` dans la boucle d'envoi de fichier de notre processus Comm. Sans cette pause, le Comm aurait envoyé tous les fragments d'un coup au Driver, saturant sa mémoire avant même que le jeton n'ait eu le temps de faire le tour. Si l'anneau comporte beaucoup de machines, le temps de latence pour qu'un fichier arrive à destination devient inacceptable, et pendant ce temps, la bande passante globale est monopolisée.

2. Quelle structure envisageriez-vous pour ce type d'application ?

Pour une application moderne nécessitant du tchat textuel et du transfert de gros fichiers, nous envisagerions une architecture hybride combinant l'anneau et le pair-à-pair.

- Pour le contrôle : Nous conserverions la structure en anneau à jeton uniquement pour échanger les messages textes, surveiller qui est connecté, et envoyer des requêtes de signalisation.
- Pour les données : Si la machine A veut envoyer un fichier lourd à la machine C, A utilise l'anneau pour demander à C : "Es-tu prêt à recevoir un fichier ?". La machine C répond "Oui" et ouvre un port d'écoute temporaire. La machine A crée alors une socket TCP directe vers la machine C, totalement en dehors de l'anneau.

Le fichier transite ainsi à vitesse maximale sur un lien direct, libérant immédiatement l'anneau pour que les autres processus Driver et Comm continuent de s'échanger des messages sans être ralentis.

Partie 6 : Jeu d'essai et Validation

Afin de valider notre implémentation de manière réaliste, nous avons déployé notre solution sur un environnement de test composé de trois machines virtuelles Linux d'adresses IP respectives 192.168.1.1, 192.168.1.2 et 192.168.1.5.

Note technique : Lors de nos premiers tests, un comportement anormal de boucle locale s'est produit. L'appel à la primitive `gethostbyname()` renvoyait l'adresse de bouclage 127.0.1.1 au lieu de l'adresse réseau de la machine, car Ubuntu configure par défaut cette association dans son fichier `/etc/hosts`. Nous avons donc dû reconfigurer le fichier `/etc/hosts` de chaque machine virtuelle avec sa véritable IP pour que le routage sur l'anneau fonctionne correctement.

Voici les résultats de nos tests pour les fonctionnalités principales :

6.1. Initialisation de l'anneau et découverte de la topologie

Dans ce premier test, nous avons lancé les processus Driver sur les trois machines pour former l'anneau, puis ouvert les interfaces Comm. Après avoir généré le tout premier jeton pour amorcer le réseau, nous avons demandé l'état de l'anneau.

```
DRIVER [192.168.1.1] en attente sur port G:8000...  
[OK] Connecte au voisin droit 192.168.1.2:8000  
[ANNEAU] Nouveau voisin gauche : 192.168.1.5 (Port d'ecoute : 8000)
```

Figure 6 : Validation de l'insertion dynamique des nœuds et découverte de la topologie avec extraction exacte des ports réseau.

6.2. Routage des messages textuels

Nous avons testé ici la capacité du Driver à lire l'en-tête du paquet pour router l'information. La machine .1 envoie un message direct à la machine .5, puis un message de diffusion destiné à l'ensemble du réseau.

```
--- INTERFACE ANNEAU ---  
1. Message Direct  
2. Diffusion (Tous)  
3. Etat de l'anneau (Graphique)  
4. Envoyer Fichier  
5. Regenerer Jeton (SOS)  
6. Quitter  
Choix : 1  
IP Destination : 192.168.1.5  
Message : Salut
```

Figure 7 : Vérification du bon routage des messages sans duplication au sein de l'anneau.

6.3. Transfert de fichier et fragmentation logicielle

C'est le test critique de notre système de contrôle de flux. Nous avons envoyé un fichier texte depuis une machine vers une autre. Le processus Comm expéditeur fragmente le fichier, et le Comm récepteur le reconstitue à la volée. Un affichage dans la console confirme le début et la fin de la réception.

```

--- INTERFACE ANNEAU ---
1. Message Direct
2. Diffusion (Tous)
3. Etat de l'anneau (Graphique)
4. Envoyer Fichier
5. Regenerer Jeton (SOS)
6. Quitter
Choix : 4
Chemin du fichier : test1.txt
IP Cible : 192.168.1.5
[FICHIER] Preparation de l'envoi...
[FICHIER] Envoi termine.

```

```

--- INTERFACE ANNEAU ---
1. Message Direct
2. Diffusion (Tous)
3. Etat de l'anneau (Graphique)
4. Envoyer Fichier
5. Regenerer Jeton (SOS)
6. Quitter
Choix :
[FICHIER] Reception terminee avec succes : recu_test1.txt

```

Figure 8 et 9 : Validation de la fragmentation logicielle lors d'un transfert binaire et conservation de l'intégrité des données.

6.4. Déconnexion coopérative et cicatrisation

Ce test vérifie la robustesse de l'anneau. La machine 192.168.1.2 sélectionne l'option 6 ("Quitter"). Elle envoie un paquet TYPE_BYE contenant l'adresse de son voisin droit (.5). La machine .1 intercepte ce paquet, ferme sa connexion avec la .2 et se connecte automatiquement à la .5.

```

[DECONNEXION] Mon voisin droit s'en va. Reconnexion vers 192.168.1.5:8000
[OK] Connecte au voisin droit 192.168.1.5:8000

```

Figure 10 : Cicatrisation automatique de l'anneau suite à la déconnexion volontaire d'un nœud, garantissant la continuité du service.

6.5. Tolérance aux pannes : La régénération du jeton

Enfin, pour simuler un blocage du réseau dû à la perte du jeton (par exemple suite à une coupure brutale), nous utilisons l'option de secours de l'IHM. La commande "Régénérer Jeton (SOS)" injecte manuellement un paquet TYPE_JETON pour relancer le circuit, démontrant ainsi la capacité du système à reprendre son fonctionnement après un incident majeur.

```

--- INTERFACE ANNEAU ---
1. Message Direct
2. Diffusion (Tous)
3. Etat de l'anneau (Graphique)
4. Envoyer Fichier
5. Regenerer Jeton (SOS)
6. Quitter
Choix : 5

```

```

--- INTERFACE ANNEAU ---
1. Message Direct
2. Diffusion (Tous)
3. Etat de l'anneau (Graphique)
4. Envoyer Fichier
5. Regenerer Jeton (SOS)
6. Quitter
Choix : 3

```

===== TOPOLOGIE DE L'ANNEAU =====

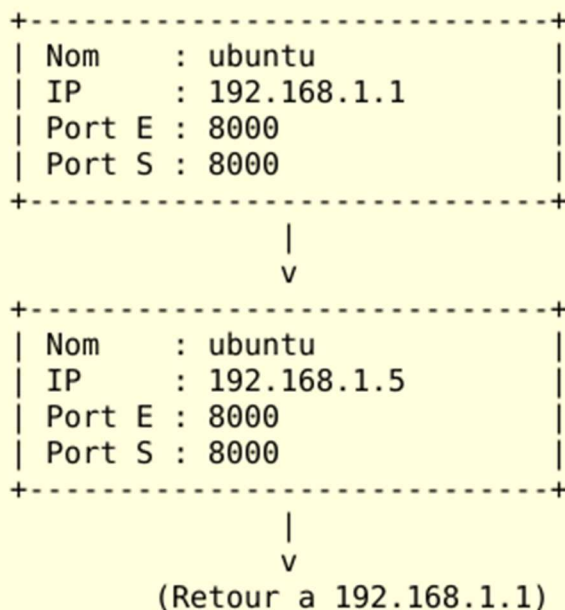


Figure 11 et 12 : Déblocage manuel du réseau via la réinjection d'un jeton de contrôle.

Conclusion

Ce projet de simulation réseau a été formateur. Nous avons réussi à implémenter un réseau en anneau dynamique, capable de supporter l'insertion de nouveaux nœuds à la volée et de gérer la fermeture de l'anneau en cas de départ d'une machine.

La contrainte d'utiliser deux processus séparés nous a obligés à bien structurer nos communications locales. Sur le plan technique, nous avons particulièrement renforcé nos compétences en programmation système C, notamment :

- La maîtrise du `select()`, indispensable pour écouter le réseau et le clavier simultanément sans bloquer le programme.
- La manipulation de l'API Sockets réseau.
- La compréhension critique justifiant l'utilisation de `ntohs()` et `htons()`.

Malgré la vulnérabilité théorique de l'anneau aux pannes matérielles, nos mécanismes de déconnexion coopérative et de régénération manuelle du jeton ont permis de rendre notre application fonctionnelle et robuste.

Annexe : Le code source complet de notre projet est fourni dans le dossier jointe à ce rapport. Il se compose de trois fichiers, qui ont été commentés pour faciliter la lecture :

- `protocole.h` : Contient la définition des constantes et de la structure de données Paquet commune aux deux processus.
- `driver.c` : Contient le code du processus gérant la topologie de l'anneau, le routage et le passage du jeton.
- `comm.c` : Contient le code de l'Interface Homme-Machine, gérant la saisie utilisateur, l'affichage de la topologie et la fragmentation logicielle pour l'envoi de fichiers.